

CSE225 — Binary Search Trees, Heaps & Priority Queues

Multiple Choice Question Bank | Built strictly from Lecture 11 (BST Part 1), Lecture 12 (BST Part 2) and Lecture 13 (Heaps & Priority Queues)

225 questions — full answer key with explanations at the end

How to use: attempt each section closed-book, then check the answer key. Every fact tested here appears on a slide — definitions, code fragments, keywords, pre/postconditions and time complexities.

Section A — Motivation and Tree Terminology

1. In an *unsorted list*, what is the complexity of `RetrieveItem`?
(a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$
2. In a *sorted list*, what is the complexity of `RetrieveItem`?
(a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N^2)$
3. In an *unsorted list*, what is the complexity of `InsertItem`?
(a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$
4. In a *sorted list*, `InsertItem` costs $O(N)$. Which operation did the lecture identify as the one we want to improve, motivating the binary search tree?
(a) `RetrieveItem` on the unsorted list (b) `InsertItem` on the sorted list (c) `InsertItem` on the unsorted list (d) `RetrieveItem` on the sorted list
5. A tree is described as a collection of nodes linked to each other, similar to which structure?
(a) Arrays (b) Linked lists (c) Stacks (d) Hash tables
6. In a tree, how many children can a node have?
(a) Exactly 2 (b) At most 1 (c) 0 or more (d) Exactly 1
7. How many parents does every node of a tree have, except the root?
(a) Zero (b) Exactly one (c) At most two (d) Any number
8. The rule "each node except the root has exactly one parent" directly implies which property?
(a) The tree is always balanced (b) There is exactly one path from the root to any other node (c) Every node is a leaf (d) The tree is complete
9. Another name used in the slides for a *child* node is:
(a) Predecessor node (b) Successor node (c) Sibling node (d) Interior node
10. Another name used in the slides for a *parent* node is:
(a) Successor node (b) Predecessor node (c) Ancestor leaf (d) Root pointer
11. Definition: the **root** is the node that has
(a) no child (b) no parent (c) two children (d) the largest value
12. Definition: a **leaf** is a node that has
(a) no parent (b) no child (c) exactly one child (d) two children
13. **Interior** nodes are defined as:
(a) Leaf nodes only (b) Non-leaf nodes (c) The root only (d) Nodes at the last level
14. The **level** of a node is defined as:
(a) The number of its children (b) The number of ancestors up to the root (distance from the root) (c) The number of leaves under it (d) The number of levels in the tree
15. The **height** of a tree, as defined in the lecture, is:
(a) The number of levels (b) The number of nodes (c) The number of leaves (d) The level of the root
16. In the lecture diagram, a tree with Level 0, Level 1 and Level 2 has height:
(a) 2 (b) 3 (c) 4 (d) 7
17. What level does the root of a tree occupy?
(a) Level 1 (b) Level 0 (c) Level h (d) Level -1

18. The slide notes that some books define height differently, namely as:
(a) The number of leaves (b) The highest level in the tree (c) Half of the node count (d) $\log_2 N$
19. A structure in which some node has two parents (two incoming arrows) is:
(a) Still a tree (b) Not a tree (c) A binary tree (d) A complete tree

Section B — Binary Trees, Full/Complete, Height Mathematics

20. A binary tree is a tree in which each node has:
(a) 0–2 children (b) exactly 2 children (c) 0 or more children (d) at most 3 children
21. Which of the following is NOT one of the four child configurations listed for a binary tree node?
(a) No child (b) One left child (c) One right child (d) Three children
22. Every node of a binary tree is the parent of two smaller trees called:
(a) Branches (b) Subtrees (c) Forests (d) Leaves
23. A binary tree is **full** if each node is:
(a) either a leaf or possesses exactly two child nodes (b) at the last level (c) either a leaf or possesses one child (d) an interior node
24. A binary tree is **complete** if:
(a) every node has two children (b) all levels (except possibly the last) are completely full, and the last level has all its nodes on the left side (c) it has 2^h nodes (d) it is a binary search tree
25. In a complete tree, the nodes of the last level must be pushed to which side?
(a) Right side (b) Left side (c) Either side (d) Centre
26. Can a binary tree be full but NOT complete?
(a) No, full implies complete (b) Yes, the lecture shows such an example (c) Only if it has one node (d) Only for min-heaps
27. Can a binary tree be complete but NOT full?
(a) No (b) Yes, the lecture shows such an example (c) Only for empty trees (d) Only when height = 1
28. The maximum number of nodes at level i of a binary tree is:
(a) $2i$ (b) 2^i (c) 2^{i-1} (d) i^2
29. The maximum number of nodes at Level 2 of a binary tree is:
(a) 2 (b) 3 (c) 4 (d) 8
30. A binary tree of height h contains the levels:
(a) 1, 2, ..., h (b) 0, 1, 2, ..., $(h-1)$ (c) 0, 1, 2, ..., h (d) 0 only
31. The maximum number of nodes N in a binary tree of height h is:
(a) 2^h (b) 2^{h-1} (c) $2^{h-1}-1$ (d) h^2-1
32. The formula $N = 2^0 + 2^1 + \dots + 2^{h-1}$ is evaluated in the slides using the sum of a:
(a) Arithmetic series (b) Geometric series (c) Harmonic series (d) Fibonacci series
33. Solving $N = 2^h - 1$ for h gives the height of a binary tree with N nodes as:
(a) $h = \log_2(N)$ (b) $h = \log_2(N+1)$ (c) $h = \log_2(N-1)$ (d) $h = N/2$
34. What function is applied to $\log_2(N+1)$ to obtain the minimum height?
(a) floor() (b) ceil() (c) round() (d) abs()
35. The **maximum** possible height of a binary tree with N nodes is:
(a) $\log_2 N$ (b) N (c) $N/2$ (d) 2^N
36. For $N = 10$ nodes, the minimum height computed in the slides is:
(a) 3.46, i.e. about 4 (b) 10 (c) 5 (d) 2
37. For $N = 10$ nodes, the maximum height is:
(a) 4 (b) 5 (c) 10 (d) 9
38. Minimum height occurs when the tree is:
(a) complete (best case) (b) skewed (worst case) (c) empty (d) full only

39. Maximum height occurs when:

- (a) the tree is complete (b) each node has one child (worst case) (c) each node has two children (d) the tree has one node

Section C — Binary Search Tree Property and Specification

40. In a BST, the value at any node is:

- (a) smaller than its left child (b) larger than the value of its left child (c) equal to its right child (d) unrelated to its children

41. In a BST, the value at any node compared to its right child is:

- (a) larger (b) smaller or equal (c) equal (d) unpredictable

42. "Larger than the value of the left child" means, more strongly, that the node value is larger than:

- (a) only that one child (b) all values in the left subtree (c) all values in the right subtree (d) all values in the tree

43. The **Structure** clause of the BST specification says the key of an element is greater than the key of any element in its ____ and less than the key of any element in its ____.

- (a) right subtree; left subtree (b) left subtree; right subtree (c) parent; child (d) sibling; parent

44. Consider the tree: root 10, left child 5, right child 45; 5 has children 2 and 25; 45 has left child 30. Is it a BST?

- (a) Yes (b) No, because 25 lies in the left subtree of 10 (c) No, because $45 > 10$ (d) Yes, because it is complete

45. Which operation has the postcondition "Tree exists and is empty"?

- (a) IsEmpty (b) MakeEmpty (c) DeleteItem (d) ResetTree

46. Boolean `IsFull` determines whether the tree is full and returns:

- (a) the number of free nodes (b) true if tree is full and false otherwise (c) true if the tree is a full binary tree (d) the height of the tree

47. Careful: in the specification of `IsFull`, "full" means

- (a) every node has two children (b) no more memory can be allocated for a new node (c) the last level is complete (d) the tree is balanced

48. `int LengthIs` returns:

- (a) the height of the tree (b) the number of elements in the tree (c) the number of leaves (d) the number of levels

49. The precondition of `RetrieveItem(ItemType& item, Boolean& found)` is:

- (a) Tree is not full (b) Key member of item is initialized (c) Tree is not empty (d) item is in tree

50. If `RetrieveItem` does not find the key, its postcondition states:

- (a) `found = false`, item is unchanged, tree is unchanged (b) `found = false` and item is set to NULL (c) the item is inserted (d) an exception is thrown

51. The precondition of `InsertItem(ItemType item)` is:

- (a) Tree is not empty (b) Tree is not full and item is not in tree (c) item is in tree (d) Key member is uninitialized

52. The postcondition of `InsertItem` is:

- (a) item is in tree; binary search property is maintained (b) tree is balanced (c) tree is complete (d) item is at the root

53. The precondition of `DeleteItem(ItemType item)` requires:

- (a) two or more matching keys (b) key member of item is initialized and one and only one element in tree has a key matching item's key (c) the tree is complete (d) the item is at a leaf

54. The postcondition of `DeleteItem` is:

- (a) No element in tree has a key matching item's key (b) The tree is empty (c) `found = false` (d) The item becomes the root

55. `Print()` prints the values in the tree in:

- (a) descending key order (b) ascending key order (c) preorder (d) level order

56. `ResetTree(OrderType order)` has the postcondition:

- (a) Tree is empty (b) Current position is prior to root of tree (c) Current position is the last node (d) The tree is traversed and printed

57. The precondition of `GetNextItem` is:

- (a) Current position is defined and the element at current position is not last in tree (b) Tree is full (c) The tree is empty (d) `order == IN_ORDER`

58. In `GetNextItem`, the flag `finished` is set to:

- (a) true if the item was found (b) (current position is last in tree) (c) true if the tree is empty (d) the order type

Section D — Node Structure, Header File and O(1) Members

59. Which declaration matches the `TreeNode` used in the lectures?

- (a) `struct TreeNode { ItemType info; TreeNode *next; };`
(b) `struct TreeNode { ItemType info; TreeNode *left, *right; };`
(c) `struct TreeNode { ItemType *info; TreeNode left, right; };`
(d) `class TreeNode { int info; };`

60. The three fields of a tree node, in the order drawn on the slide box, are:

- (a) info, left, right (b) left, info, right (c) right, info, left (d) left, right, info

61. In `binarysearchtree.h`, `ItemType` is defined with:

- (a) `typedef int ItemType;` (b) `typedef char ItemType;` (c) `typedef float ItemType;` (d) `template<class ItemType>`

62. The enumeration declared in the header is:

- (a) `enum OrderType {PRE_ORDER, IN_ORDER, POST_ORDER};`
(b) `enum OrderType {ASCENDING, DESCENDING};`
(c) `enum Order {LEFT, RIGHT};`
(d) `enum OrderType {IN_ORDER, LEVEL_ORDER};`

63. The only private data member of class `TreeType` in Lecture 11 is:

- (a) `TreeNode node;` (b) `TreeNode* root;` (c) `int length;` (d) `ItemType* elements;`

64. Which include guard is used in the header file?

- (a) `#ifndef TREE_H` (b) `#ifndef BINARYSEARCHTREE_H_INCLUDED` (c) `#pragma once` (d) `#ifndef BST_INCLUDED`

65. The `TreeType` constructor body is:

- (a) `root = new TreeNode;` (b) `root = NULL;` (c) `root->info = 0;` (d) `length = 0;`

66. `bool TreeType::IsEmpty()` is implemented as:

- (a) `return root == NULL;` (b) `return length == 0;` (c) `return root != NULL;` (d) `return CountNodes(root) == 0;`

67. What header is included in `binarysearchtree.cpp` to support the `IsFull` implementation?

- (a) `<iostream>` (b) `<new>` (c) `<memory>` (d) `<exception>`

68. In `IsFull()`, which exception type is caught?

- (a) `std::out_of_range` (b) `std::bad_alloc` (c) `FullPQ` (d) `std::runtime_error`

69. Inside the `try` block of `IsFull()`, after `location = new TreeNode;` the next statements are:

- (a) `return true;` (b) `delete location; return false;` (c) `root = location; return false;` (d) `delete location; return true;`

70. The time complexities of `TreeType()`, `IsEmpty()` and `IsFull()` are respectively:

- (a) $O(1)$, $O(1)$, $O(1)$ (b) $O(1)$, $O(N)$, $O(N)$ (c) $O(N)$, $O(1)$, $O(\log N)$ (d) $O(\log N)$, $O(\log N)$, $O(1)$

71. In the pointer-vs-reference demo, `void f(int *addr)` is called with `y`. After the call, the printed value of `y` is:

- (a) changed (b) unchanged, because the pointer is passed by value (c) `NULL` (d) undefined

72. In the same demo, `void f(int *&addr)` causes:

- (a) no change to `y` (b) `y` to change, because the pointer is passed by reference (c) a compile error (d) a memory leak only

73. Why must `Insert` take `TreeNode *&tree` rather than `TreeNode *tree`?

- (a) To save memory (b) So the assignment `tree = new TreeNode;` actually updates the caller's pointer (parent's link or root) (c) Because `ItemType` is `char` (d) To allow recursion

Section E — Function `InsertItem`

74. What is the base case of the recursive `Insert` function?

- (a) `tree->left == NULL` (b) `tree == NULL`, i.e. the insertion place is found (c) `item == tree->info` (d) The tree is full

75. In the base case, which four assignments are performed?

- (a) `tree = new TreeNode; tree->right = NULL; tree->left = NULL; tree->info = item;`
- (b) `tree->info = item;` only
- (c) `tree = NULL; delete tree;`
- (d) `root = new TreeNode; root->info = item;`

76. "General case 1" of `Insert` is taken when:

- (a) `item > tree->info`
- (b) `item < tree->info`, insert in the left subtree
- (c) `tree == NULL`
- (d) `item == tree->info`

77. "General case 2" of `Insert` performs:

- (a) `Insert(tree->left, item);`
- (b) `Insert(tree->right, item);`
- (c) `delete tree;`
- (d) `return;`

78. The member function `TreeType::InsertItem(ItemType item)` body is:

- (a) `Insert(root, item);`
- (b) `Insert(item);`
- (c) `root = Insert(item);`
- (d) `Insert(root->left, item);`

79. Worst-case complexity of `InsertItem` is:

- (a) $O(1)$
- (b) $O(\log N)$
- (c) $O(N)$
- (d) $O(N \log N)$

80. Best-case complexity of `InsertItem` is:

- (a) $O(1)$
- (b) $O(\log N)$
- (c) $O(N)$
- (d) $O(N^2)$

81. Average-case complexity of `InsertItem` is:

- (a) $O(1)$
- (b) $O(\lg N)$
- (c) $O(N)$
- (d) $O(N \lg N)$

82. Inserting 5, 9, 7, 3, 8, 12, 6, 4, 20 in that order into an empty BST puts which value at the root?

- (a) 3
- (b) 5
- (c) 9
- (d) 20

83. Continuing Q82, node 7 ends up as the:

- (a) left child of 9
- (b) right child of 9
- (c) left child of 5
- (d) root

84. Continuing Q82, the value 20 becomes the:

- (a) right child of 12
- (b) left child of 12
- (c) right child of 9
- (d) left child of 8

85. During `InsertItem(7)` on the tree {5, 9}, the chain of calls shown was `Insert(root,7) → Insert(root->right,7) → Insert(root->right->left,7)`. Which cases were used in order?

- (a) Base, general 1, general 2
- (b) General case 2, general case 1, base case
- (c) General 1, general 1, base
- (d) Base, base, base

86. Inserting the input sequence A B C D E F G into an empty BST produces:

- (a) a complete tree
- (b) a full tree
- (c) a skewed chain of height 7
- (d) a tree of height 3

87. Inserting D B F A C E G produces:

- (a) a linear chain
- (b) a short, balanced tree (D at root with B and F as children)
- (c) an empty tree
- (d) a max-heap

88. What does the "Impact of order of insertion" slide demonstrate?

- (a) That insertion order does not matter
- (b) That the same set of items produces trees of different heights depending on insertion order
- (c) That BSTs are always complete
- (d) That duplicates are allowed

Section F — Function Lengths (`CountNodes`)

89. `LengthIs` calculates the number of nodes:

- (a) iteratively with a loop
- (b) recursively
- (c) from a stored counter
- (d) using a queue

90. The base case of `CountNodes` is: if the tree is empty then length =

- (a) 1
- (b) 0
- (c) -1
- (d) NULL

91. The general case of `CountNodes` returns:

- (a) left count + right count
- (b) left count + right count + 1
- (c) $\max(\text{left}, \text{right}) + 1$
- (d) $2 \times \text{left count}$

92. Which line is the correct general case in code?

- (a) `return CountNodes(tree->left) + CountNodes(tree->right) + 1;`
- (b) `return CountNodes(tree->left) + CountNodes(tree->right);`
- (c) `return 1 + CountNodes(tree);`
- (d) `return CountNodes(root) + 1;`

93. `int TreeType::LengthIs()` is implemented as:
 (a) `return CountNodes(root);` (b) `return CountNodes(root->left);` (c) `return length;` (d) `CountNodes(root);`
94. The time complexity of `LengthIs` is:
 (a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N^2)$
95. Why is `LengthIs` $O(N)$ and not $O(\log N)$?
 (a) Because it uses a queue (b) Because every node must be visited (c) Because the tree is skewed (d) Because it allocates memory
96. In the traced example, the counts returned by node 3's left and right calls were 0 and 1, so node 3's subtree contributed:
 (a) 1 (b) 2 (c) 3 (d) 0
97. In the same trace on the tree {5,3,4,9,7,6,8,12,20}, the final value returned at the root was:
 (a) 8 (b) 9 (c) 10 (d) 6

Section G — Function RetrieveItem

98. `RetrieveItem` finds the target node:
 (a) iteratively (b) recursively (c) by scanning an array (d) via a queue
99. Base case 1 of `Retrieve` is:
 (a) `item == current node info` → item found (b) tree is empty → item is not found (c) `item < info` (d) `item > info`
100. Base case 2 of `Retrieve` is:
 (a) tree is empty (b) `item == current_node_info`, i.e. item is found (c) `item > info` (d) `found == false`
101. General case 1 of `Retrieve` (`item < current_node_info`) means:
 (a) search the right subtree (b) search the left subtree (c) stop (d) insert the item
102. In the `else` branch of `Retrieve` (the match), the two statements executed are:
 (a) `item = tree->info; found = true;` (b) `found = false; return;` (c) `tree->info = item; found = true;` (d) `delete tree;`
103. The signature of the helper is:
 (a) `void Retrieve(TreeNode* tree, ItemType& item, bool& found)`
 (b) `void Retrieve(TreeNode*& tree, ItemType item, bool found)`
 (c) `bool Retrieve(TreeNode* tree, ItemType item)`
 (d) `ItemType Retrieve(TreeNode* tree)`
104. Why does `Retrieve` take `TreeNode* tree` (not `TreeNode*&`) while `Insert` takes a reference to pointer?
 (a) `Retrieve` is iterative (b) `Retrieve` never modifies the tree structure, so it does not need to change the caller's pointer (c) `Retrieve` is faster this way only (d) It is a compiler requirement
105. Complexity of `RetrieveItem`: worst, best, average respectively:
 (a) $O(N)$, $O(1)$, $O(\log N)$ (b) $O(\log N)$, $O(1)$, $O(N)$ (c) $O(N)$, $O(N)$, $O(N)$ (d) $O(1)$, $O(1)$, $O(1)$
106. When does `RetrieveItem` hit its best case $O(1)$?
 (a) When the tree is empty (b) When the item is at the root (c) When the item is a leaf (d) When the tree is skewed
107. Searching for 8 in the tree {5,3,4,9,7,6,8,12,20} used the case sequence:
 (a) general 2, general 1, general 2, base case 2 (b) general 1, general 1, base case 1 (c) base case 2 only (d) general 2, general 2, base case 1
108. Searching for 10 in the same tree ends with:
 (a) base case 2 (found) (b) base case 1 (not found, `tree == NULL`) (c) an exception (d) general case 1

Section H — Function Print

109. `Print` prints the items:
 (a) in the order inserted (b) in sorted order, recursively (c) in preorder (d) in reverse order
110. The general case of `PrintTree` performs, in order:
 (a) print node, print left, print right (b) print left subtree, print current node, print right subtree (c) print left, print right, print node (d) print right, node, left

- 111.** The base case of `PrintTree` (empty tree) does:
 (a) print a blank line (b) nothing to print (c) print NULL (d) throw an exception
- 112.** Which guard does `PrintTree` use?
 (a) `if (tree == NULL)` (b) `if (tree != NULL)` (c) `while (tree != NULL)` (d) `if (root != NULL)`
- 113.** `Print` is equivalent to which traversal?
 (a) Preorder (b) Inorder (c) Postorder (d) Level order
- 114.** The output of `Print()` on the tree {5,3,4,9,7,6,8,12,20} is:
 (a) 5 3 4 9 7 6 8 12 20 (b) 3 4 5 6 7 8 9 12 20 (c) 4 3 6 8 7 20 12 9 5 (d) 20 12 9 8 7 6 5 4 3
- 115.** The complexity of `Print` is:
 (a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$

Section I — Deleting a Node

- 116.** In the `DeleteNode` code, how many `if / else-if` branches are written to cover deletion?
 (a) 2 (b) 3 (c) 4 (d) 5
- 117.** To delete a **leaf** node, the code sets:
 (a) `tree = tree->left;` (b) `tree = NULL; delete tempPtr;` (c) `tree = tree->right;` (d) `GetPredecessor(...)`
- 118.** If the node to delete has only a right child (`tree->left == NULL`), the code executes:
 (a) `tree = tree->right; delete tempPtr;` (b) `tree = tree->left; delete tempPtr;` (c) `tree = NULL;` (d) `Delete(tree->right, data);`
- 119.** If `tree->right == NULL` (only a left child), the code executes:
 (a) `tree = tree->right;` (b) `tree = tree->left; delete tempPtr;` (c) `tree = NULL;` (d) nothing
- 120.** A node with two children is replaced by its:
 (a) successor (leftmost node of the right subtree) (b) predecessor (rightmost node in the left subtree) (c) parent (d) left child
- 121.** The predecessor of a node is found by:
 (a) going left once, then left repeatedly (b) going left once, then right until right is NULL (c) going right once, then left repeatedly (d) taking the root
- 122.** Which is the correct body of `GetPredecessor(TreeNode* tree, ItemType& data)?`
 (a) `while (tree->left != NULL) tree = tree->left; data = tree->info;`
 (b) `while (tree->right != NULL) tree = tree->right; data = tree->info;`
 (c) `if (tree->right != NULL) GetPredecessor(tree->right, data);`
 (d) `data = tree->left->info;`
- 123.** In the two-children case, after `GetPredecessor(tree->left, data); tree->info = data;` the next statement is:
 (a) `delete tree;` (b) `Delete(tree->left, data);` (c) `Delete(tree->right, data);` (d) `tree = NULL;`
- 124.** In the example tree rooted at 9 (with 3, 4, 16, 10, 14, 13, 12, 18, 20), deleting the root 9 replaces it with:
 (a) 3 (b) 4 (c) 10 (d) 16
- 125.** In the same tree, deleting 16 replaces it with:
 (a) 13 (b) 14 (c) 18 (d) 10
- 126.** In the same tree, deleting node 18 (which has only the child 20) is handled by:
 (a) making 20 the child of 16 (b) deleting the whole subtree (c) replacing 18 with its predecessor (d) setting the link to NULL
- 127.** The role of `tempPtr` in `DeleteNode` is to:
 (a) hold the predecessor value (b) remember the node being removed so it can be deleted after re-linking (c) walk the tree (d) store the item's key
- 128.** The recursive `Delete(TreeNode*& tree, ItemType item)` calls `DeleteNode(tree)` when:
 (a) `item < tree->info` (b) `item > tree->info` (c) neither, i.e. the item matches `tree->info` (d) `tree == NULL`

129. `TreeType::DeleteItem(ItemType item)` is implemented as:

- (a) `Delete(root, item);` (b) `DeleteNode(root);` (c) `Delete(item);` (d) `GetPredecessor(root, item);`

Section J — Traversal Techniques

130. **Inorder** traversal visits:

- (a) node, left subtree, right subtree (b) left subtree, node, right subtree (c) left subtree, right subtree, node (d) right, node, left

131. **Postorder** traversal visits:

- (a) left subtree, right subtree, then the node (b) node, then left, then right (c) left, node, right (d) right, left, node

132. **Preorder** traversal visits:

- (a) left, node, right (b) the node, then the left subtree, then the right subtree (c) left, right, node (d) node, right, left

133. For the tree rooted at 9 (3, 4, 16, 10, 14, 13, 12, 18, 20), the **inorder** traversal is:

- (a) 9 3 4 16 10 14 13 12 18 20 (b) 3 4 9 10 12 13 14 16 18 20 (c) 4 3 12 13 14 10 20 18 16 9 (d) 20 18 16 14 13 12 10 9 4 3

134. For the same tree, the **postorder** traversal is:

- (a) 4 3 12 13 14 10 20 18 16 9 (b) 9 3 4 16 10 14 13 12 18 20 (c) 3 4 9 10 12 13 14 16 18 20 (d) 9 16 18 20 10 14 13 12 3 4

135. For the same tree, the **preorder** traversal is:

- (a) 3 4 9 10 12 13 14 16 18 20 (b) 9 3 4 16 10 14 13 12 18 20 (c) 4 3 12 13 14 10 20 18 16 9 (d) 9 3 16 4 10 18 14 20 13 12

136. Which traversal always outputs a BST's keys in ascending order?

- (a) Preorder (b) Inorder (c) Postorder (d) All of them

137. To support the iterator operations, class `TreeType` stores which extra private members?

- (a) three stacks (b) `QueueType preQueue, inQueue, postQueue;` (c) an array of `ItemType` (d) a single `QueueType`

138. In `InOrder(TreeNode* tree, QueueType& inQueue)`, the enqueue statement is placed:

- (a) before both recursive calls (b) between the two recursive calls (c) after both recursive calls (d) inside the base case

139. In `PreOrder`, the statement `preQueue.Enqueue(tree->info);` appears:

- (a) first, before the recursive calls (b) between the calls (c) last (d) it is not used

140. In `PostOrder`, `postQueue.Enqueue(tree->info);` appears:

- (a) first (b) between the two recursive calls (c) after both recursive calls (d) in the base case

141. `TreeType::ResetTree(OrderType order)` is implemented with:

- (a) an if-else chain over `ItemType` (b) a `switch(order)` having cases `PRE_ORDER`, `IN_ORDER`, `POST_ORDER` (c) a while loop (d) recursion on root

142. In `GetNextItem`, the variable `finished` is initialised to:

- (a) true (b) false (c) the queue length (d) NULL

143. In `GetNextItem`, when is `finished` set to true?

- (a) Always (b) When the corresponding queue becomes empty after the `Dequeue` (c) When the item is not found (d) When order is `IN_ORDER`

Section K — Priority Queues

144. A priority queue differs from an ordinary queue because it:

- (a) allows only insertion (b) does not necessarily maintain First In First Out order (c) is always sorted (d) uses a stack internally

145. In a priority queue, items can be added:

- (a) only in sorted order (b) in any order (c) only at the front (d) only when it is empty

146. Which item is always deleted from a priority queue?

- (a) The oldest item (b) The item with the highest priority (c) The newest item (d) A random item

147. The **Structure** clause of the priority queue specification says the queue is arranged to support:
(a) FIFO order (b) access to the highest-priority item (c) binary search (d) random access
148. `Enqueue(ItemType newItem)` throws which exception when the queue is full?
(a) `EmptyPQ` (b) `FullPQ` (c) `std::bad_alloc` (d) `OverflowError`
149. `Dequeue(ItemType& item)` throws which exception when the queue is empty?
(a) `FullPQ` (b) `EmptyPQ` (c) `std::bad_alloc` (d) `UnderflowPQ`
150. The postcondition of `Dequeue` states that:
(a) the lowest priority element is removed (b) the highest priority element has been removed from the queue and item is a copy of the removed element (c) the queue is empty (d) `newItem` is in the queue

Section L — Heaps: Shape and Order Properties

151. A heap is defined as:
(a) a full binary tree (b) a complete binary tree with an order property (c) a binary search tree (d) a skewed tree
152. In a **min-heap**, each element contains a value that is:
(a) greater than or equal to the value of each of its children (b) less than or equal to the value of each of its children (c) equal to its parent (d) less than its parent
153. In a **max-heap**, each element contains a value that is:
(a) less than or equal to each of its children (b) greater than or equal to the value of each of its children (c) greater than its parent (d) unrelated to its children
154. "A complete binary tree" is which property of a heap?
(a) The order property (b) The shape property (c) The search property (d) The balance property
155. "Each element is greater than or equal to the value of each of its children" is which property?
(a) The shape property (b) The order property (c) The heap property of BSTs (d) The completeness property
156. The slides state that the shape of all heaps with a given number of elements is:
(a) different (b) the same (c) always skewed (d) undefined
157. In a max-heap, the root node always contains:
(a) the smallest value (b) the largest value in the heap (c) the median value (d) the last inserted value
158. Besides the root property, the slides add that in a heap:
(a) the leaves are sorted (b) the subtrees are heaps as well (c) each level is sorted (d) it is also a BST
159. Is a heap a binary search tree?
(a) Yes, always (b) No — the ordering is between parent and children only, not left/right (c) Only a min-heap is (d) Only when complete
160. An inorder traversal of a max-heap produces:
(a) ascending order (b) descending order (c) no guaranteed sorted order (d) the level order

Section M — ReheapDown and ReheapUp

161. `ReheapDown` is used when the order property is violated by:
(a) the last leaf node only (b) the root node only (c) all nodes (d) an interior node chosen at random
162. `ReheapDown` repairs the heap by:
(a) moving the element down from the root position until the order property is satisfied (b) moving the element up (c) rebuilding the array (d) sorting the array
163. `ReheapUp` is used when the order property is violated by:
(a) the root node only (b) the last leaf node only (c) a leaf on the left (d) two nodes at once
164. `ReheapUp` repairs the heap by:
(a) moving the element down (b) moving the element up in the tree until it ends up in its correct position (c) swapping the two children (d) deleting the leaf
165. In the `ReheapDown` walkthrough, the root value 6 in the heap (6; 11, 4; 7, 9, 3, 1; 2, 5, 8) is first swapped with:
(a) 4 (b) 11 (c) 9 (d) 7
166. Continuing that walkthrough, after 6 reaches the position under 11, the next swap is with:
(a) 7 (b) 9 (c) 8 (d) 2

- 167.** When ReheapDown pushes a value down, it swaps with:
 (a) the left child always (b) the larger of the two children (for a max-heap) (c) the smaller child (d) the parent
- 168.** In the ReheapUp walkthrough, the last leaf 11 in the heap (9; 8, 4; 7, 6, 3, 1; 2, 5, 11) is swapped successively with:
 (a) 6, then 8, then 9 (b) 5, then 7, then 9 (c) 8 then 4 (d) 9 only
- 169.** The complexity of ReheapUp is:
 (a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$
- 170.** The complexity of ReheapDown is:
 (a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N^2)$
- 171.** Why are the reheap operations $O(\log N)$?
 (a) They visit every node (b) They travel one root-to-leaf path, and a complete tree's height is logarithmic in N (c) They use recursion (d) They copy the array
- 172.** The complexity of the helper Swap is:
 (a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$
- 173.** The three assignments inside Swap(ItemType& one, ItemType& two) are:
 (a) temp = one; one = two; two = temp; (b) one = two; two = one; temp = one; (c) temp = two; two = one; one = temp; only with pointers (d) one = temp; two = temp;
- 174.** In ReheapUp(int root, int bottom), the guard condition is:
 (a) if (bottom < root) (b) if (bottom > root) (c) while (root > 0) (d) if (root == 0)
- 175.** Inside ReheapUp, the parent index is computed as:
 (a) parent = bottom/2; (b) parent = (bottom-1)/2; (c) parent = 2*bottom+1; (d) parent = bottom-1;
- 176.** ReheapUp swaps when:
 (a) elements[parent] < elements[bottom] (b) elements[parent] > elements[bottom] (c) always (d) bottom == 0
- 177.** After swapping, ReheapUp recurses as:
 (a) ReheapUp(root, bottom); (b) ReheapUp(root, parent); (c) ReheapUp(parent, bottom); (d) ReheapDown(root, parent);
- 178.** In ReheapDown, the child indices are initialised as:
 (a) rightChild = root*2+2, leftChild = root*2+1; (b) rightChild = root*2+1, leftChild = root*2+2; (c) leftChild = root/2, rightChild = root/2+1; (d) leftChild = root+1, rightChild = root+2;
- 179.** In ReheapDown, the condition if (leftChild <= bottom) checks that:
 (a) the node is a leaf (b) there is at least one child (c) the heap is full (d) the root is valid
- 180.** In ReheapDown, if (leftChild == bottom) means:
 (a) there are two children (b) the left child is the only child (c) the heap is empty (d) a swap is required
- 181.** When there are two children, ReheapDown chooses maxChild by:
 (a) if (elements[leftChild] <= elements[rightChild]) maxChild = rightChild; else maxChild = leftChild;
 (b) maxChild = leftChild; always
 (c) maxChild = rightChild; always
 (d) if (elements[leftChild] > elements[rightChild]) maxChild = rightChild;
- 182.** ReheapDown performs a swap only when:
 (a) elements[root] < elements[maxChild] (b) elements[root] > elements[maxChild] (c) always (d) root == bottom
- 183.** After swapping, ReheapDown recurses as:
 (a) ReheapDown(root, bottom); (b) ReheapDown(maxChild, bottom); (c) ReheapDown(root, maxChild); (d) ReheapUp(maxChild, bottom);

Section N — Array Implementation of Heaps

184. Why do heaps not need linked structures?
(a) Because they are small (b) Because the tree is complete, so there are no "holes" in the tree (c) Because they store chars (d) Because pointers are slow
185. In the array representation, the root is stored at index:
(a) 1 (b) 0 (c) length (d) length-1
186. For a value at index i , the left child is at index:
(a) $2i$ (b) $2i+1$ (c) $2i+2$ (d) $i/2$
187. For a value at index i , the right child is at index:
(a) $2i$ (b) $2i+1$ (c) $2i+2$ (d) $(i-1)/2$
188. For a value at index i , the parent is at index:
(a) $i/2$ (b) $(i-1)/2$ (c) $2i+1$ (d) $i-2$
189. Using the slide's example array 11, 9, 4, 7, 8, 3, 1, 2, 5, 6, the children of the value 9 (index 1) are:
(a) 4 and 7 (b) 7 and 8 (c) 3 and 1 (d) 2 and 5
190. In that same array, the parent of the element at index 6 is at index:
(a) 2 (b) 3 (c) 5 (d) 12
191. The two data members declared inside `class HeapType` are:
(a) `ItemType* elements; int numElements;` (b) `ItemType elements[100]; int length;` (c) `TreeNode* root; int size;` (d) `int maxItems; int length;`
192. The two member functions declared inside `class HeapType` are:
(a) Enqueue and Dequeue (b) `ReheapDown(int root, int bottom)` and `ReheapUp(int root, int bottom)` (c) Insert and Delete (d) `MakeEmpty` and `IsFull`
193. `class HeapType` is declared as:
(a) a plain class (b) a class template with `template<class ItemType>` (c) a struct (d) an enum

Section O — PQType: Enqueue, Dequeue and Class Design

194. Which two empty exception classes are declared in `pqtype.h`?
(a) `class FullPQ{};` `class EmptyPQ{};` (b) `class Overflow{};` `class Underflow{};` (c) `class BadAlloc{};` (d) none
195. The three private members of `class PQType` are:
(a) `int length; HeapType<ItemType> items; int maxItems;` (b) `TreeNode* root; int length; int max;` (c) `ItemType* elements; int numElements; int length;` (d) `QueType preQue, inQue, postQue;`
196. The constructor of `PQType` takes:
(a) no arguments (b) an `int` (the maximum number of items) (c) an `ItemType` (d) a `HeapType`
197. The body of `PQType<ItemType>::PQType(int max)` contains:
(a) `maxItems = max; items.elements = new ItemType[max]; length = 0;`
(b) `length = max; elements = new ItemType[max];`
(c) `maxItems = max; length = max;`
(d) `items.elements = NULL;`
198. The destructor `~PQType()` executes:
(a) `delete items.elements;` (b) `delete [] items.elements;` (c) `length = 0;` (d) nothing
199. `PQType::MakeEmpty()` is implemented as:
(a) `delete [] items.elements;` (b) `length = 0;` (c) `maxItems = 0;` (d) `items.elements = NULL;`
200. `PQType::IsFull()` returns:
(a) `length == 0` (b) `length == maxItems` (c) `length > maxItems` (d) `items.elements == NULL`
201. `PQType::IsEmpty()` returns:
(a) `length == maxItems` (b) `length == 0` (c) `maxItems == 0` (d) `length < 0`
202. The complexities of the `PQType` constructor, destructor and `MakeEmpty` are:
(a) $O(1)$, $O(1)$, $O(1)$ (b) $O(N)$, $O(N)$, $O(1)$ (c) $O(\log N)$, $O(1)$, $O(1)$ (d) $O(1)$, $O(N)$, $O(N)$

- 203.** The three steps of the **Enqueue** operation, in order, are:
(a) ReheapUp, increment length, store item
(b) Increment length; add the item as the last leaf node at index `length-1`; perform ReheapUp
(c) Store at index 0; ReheapDown; increment length
(d) Decrement length; ReheapDown
- 204.** In **Enqueue**, the reheap call is exactly:
(a) `items.ReheapUp(0, length-1)`; (b) `items.ReheapUp(0, length)`; (c) `items.ReheapDown(0, length-1)`; (d) `items.ReheapUp(length-1, 0)`;
- 205.** The first statement of **Enqueue** is a test:
(a) `if (length == 0) throw EmptyPQ();` (b) `if (length == maxItems) throw FullPQ();` (c) `if (items.elements == NULL) return;` (d) `if (length < 0) throw FullPQ();`
- 206.** The complexity of **Enqueue** is:
(a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$
- 207.** In the **Enqueue-15** walkthrough (heap 11, 9, 4, 7, 8, 3, 1, 2, 5, 6), 15 is first placed at index:
(a) 0 (b) 9 (c) 10 (d) 11
- 208.** In that walkthrough, 15 is swapped with which sequence of values on its way up?
(a) 6, 9, 11 (b) 8, 9, 11 (c) 4, 9, 11 (d) 8, 4, 11
- 209.** After enqueueing 15, the new root of the heap is:
(a) 11 (b) 15 (c) 9 (d) 8
- 210.** The three steps of the **Dequeue** operation, in order, are:
(a) Replace the root with the last leaf node; decrement length; perform ReheapDown
(b) ReheapUp; decrement length; delete the root
(c) Delete the last leaf; ReheapDown; decrement length
(d) Swap the two children; decrement length
- 211.** Which statement in **Dequeue** saves the returned value?
(a) `item = items.elements[length-1];` (b) `item = items.elements[0];` (c) `item = items.elements[length];` (d) `return items.elements[0];`
- 212.** The reheap call inside **Dequeue** is exactly:
(a) `items.ReheapDown(0, length-1)`; (b) `items.ReheapUp(0, length-1)`; (c) `items.ReheapDown(0, length)`; (d) `items.ReheapDown(length-1, 0)`;
- 213.** In **Dequeue**, the order of the statements is:
(a) `item = elements[0]; elements[0] = elements[length-1]; length--;` `ReheapDown(0, length-1);`
(b) `length--;` `item = elements[0];` `ReheapDown(0, length);`
(c) `ReheapDown(0, length-1);` `item = elements[0];` `length--;`
(d) `elements[0] = elements[length];` `length--;`
- 214.** The complexity of **Dequeue** is:
(a) $O(1)$ (b) $O(\log N)$ (c) $O(N)$ (d) $O(N \log N)$
- 215.** In the **Dequeue** walkthrough on heap 11, 9, 4, 7, 8, 3, 1, 2, 5, 6, the value returned is:
(a) 6 (b) 11 (c) 9 (d) 1
- 216.** In that walkthrough, which value is moved into the root position before **ReheapDown**?
(a) 9 (b) 6 (the last leaf) (c) 4 (d) 5
- 217.** After the **Dequeue**, **length** becomes:
(a) 10 (b) 9 (c) 11 (d) 8
- 218.** After the **ReheapDown** in that walkthrough, the resulting array is:
(a) 9 8 4 7 6 3 1 2 5 (b) 6 9 4 7 8 3 1 2 5 (c) 9 6 4 7 8 3 1 2 5 (d) 11 9 4 7 8 3 1 2 5
- 219.** Why must **ReheapDown** be called with `length-1` rather than the old length?
(a) To save one comparison (b) Because the last leaf node is now out of the tree (c) Because indices start at 1
(d) Because the heap is a min-heap
- 220.** Which single header is included by `pqtype.h` to obtain the heap?
(a) `"binarysearchtree.h"` (b) `"heap.h"` (c) `<new>` (d) `<queue>`

Section P — Mixed Complexity Review

- 221.** Which pair of BST operations both have average complexity $O(\log N)$?
 (a) InsertItem and RetrieveItem (b) Print and Lengths (c) Lengths and IsFull (d) Print and InsertItem
- 222.** Which two BST operations are $O(N)$ in every case because they must visit every node?
 (a) InsertItem and RetrieveItem (b) Lengths and Print (c) IsEmpty and IsFull (d) MakeEmpty and DeleteItem
- 223.** Which BST operation is $O(1)$ regardless of the tree?
 (a) Lengths (b) IsEmpty (c) Print (d) DeleteItem
- 224.** The worst case $O(N)$ of search/insert in a BST corresponds to a tree that is:
 (a) complete (b) skewed, i.e. every node has one child (c) full (d) empty
- 225.** Which structure guarantees $O(\log N)$ for both of its main operations because its shape is always complete?
 (a) A binary search tree (b) A heap-based priority queue (c) A sorted list (d) An unsorted list

ANSWER KEY — with one-line justifications

1	c	Unsorted list must be scanned linearly.
2	b	Sorted list allows binary search.
3	a	Unsorted list can insert at the front/end directly.
4	b	InsertItem on a sorted list is $O(N)$ — the slide asks "Can we improve this operation?"
5	b	"Collection of nodes linked to each other (similar to linked lists)".
6	c	"Each node can have 0 or more children (successor nodes)".
7	b	"Each node (except the root) has exactly one parent".
8	b	Exactly one parent per node gives exactly one root-to-node path.
9	b	Children = successor nodes.
10	b	Parent = predecessor node.
11	b	Root $\Rightarrow$ node with no parent.
12	b	Leaf $\Rightarrow$ node(s) with no child.
13	b	Interior $\Rightarrow$ non-leaf nodes.
14	b	Level = number of ancestors up to the root = distance from root.
15	a	Height $\Rightarrow$ number of levels.
16	b	Levels 0, 1, 2 $\Rightarrow$ 3 levels $\Rightarrow$ height 3.
17	b	The root is at Level 0.
18	b	"Some books use height = highest level in the tree".
19	b	A node with two parents violates the one-parent rule (the "Not Tree" figure).
20	a	"Tree with 0 – 2 children per node".
21	d	Only four cases: none, left only, right only, both.
22	b	Left subtree and right subtree.
23	a	Full: every node is a leaf or has exactly two children.
24	b	Exact wording of the complete-tree definition.
25	b	Last level filled from the left.
26	b	The slide labels one figure "Full but not complete".
27	b	The slide labels one figure "Not full but complete".
28	b	Maximum nodes in level $i = 2^i$, $i = 0, 1, 2, \dots$
29	c	$2^2 = 4$.
30	b	"We have the levels 0, 1, 2 ... (h-1)".
31	b	$N = 2^0 + \dots + 2^{h-1} = (2^h - 1) / (2 - 1) = 2^h - 1$.
32	b	Powers of two form a geometric series.

33	b	From $N = 2^h - 1 \Rightarrow h = \log_2(N+1)$.
34	b	"Minimum height is $\log_2(N+1)$ (apply ceil() to the result)".
35	b	Maximum height is N (one node per level).
36	a	$\log(10+1) = 3.46 \sim 4$.
37	c	Maximum height = 10.
38	a	"When tree is complete (best case)".
39	b	"When each node has one child (worst case)".
40	b	Value at any node is larger than the value of its left child.
41	b	Value at any node is smaller than or equal to the value of its right child.
42	b	Larger than the left child $\Rightarrow$ larger than all values in the left subtree.
43	b	Structure clause: greater than every key in its left subtree, less than every key in its right subtree.
44	b	25 sits in 10's left subtree but $25 > 10$ — the slide marks this tree "Not a binary search tree".
45	b	MakeEmpty postcondition: "Tree exists and is empty".
46	b	Returns true if tree is full and false otherwise.
47	b	IsFull tries new TreeNode and reports full when allocation fails.
48	b	LengthIs determines / returns the number of elements in tree.
49	b	Precondition: "Key member of item is initialized".
50	a	found = false, item unchanged, tree unchanged.
51	b	Precondition: "Tree is not full. item is not in tree".
52	a	Postcondition: "item is in tree. Binary search property is maintained".
53	b	One and only one element in tree has a key matching item's key.
54	a	No element in tree has a key matching item's key.
55	b	Print outputs values in ascending key order.
56	b	ResetTree postcondition: "Current position is prior to root of tree".
57	a	Current position is defined; element at current position is not last in tree.
58	b	finished = (current position is last in tree).
59	b	<code>struct TreeNode { ItemType info; TreeNode *left, *right; };</code>
60	b	The box on the slide is drawn as left info right.
61	b	<code>typedef char ItemType;</code>
62	a	<code>enum OrderType {PRE_ORDER, IN_ORDER, POST_ORDER};</code>
63	b	<code>private: TreeNode* root;</code>
64	b	<code>#ifndef BINARYSEARCHTREE_H_INCLUDED</code>
65	b	<code>TreeType::TreeType() { root = NULL; }</code>
66	a	<code>return root == NULL;</code>
67	b	<code>#include <new> for std::bad_alloc.</code>
68	b	<code>catch(std::bad_alloc exception) { return true; }</code>
69	b	Allocate, then delete location; return false; — memory was available.
70	a	All three are $O(1)$.
71	b	A plain pointer parameter is a copy; the caller's pointer is unaffected (both prints show 0x28ff18).
72	b	With <code>int *&addr</code> the caller's pointer itself is modified (0x28ff0c $\rightarrow$ 0x28fee4).
73	b	Only a reference-to-pointer lets the new node be attached to the parent's link (or to root).
74	b	<code>if (tree == NULL) // Base case: Insertion place found.</code>
75	a	Allocate, set right = NULL, left = NULL, info = item.
76	b	<code>else if (item < tree->info) Insert(tree->left, item); // General case 1</code>
77	b	<code>else Insert(tree->right, item); // General case 2</code>

78	a	<code>void TreeType::InsertItem(ItemType item) { Insert(root, item); }</code>
79	c	Worst case: $O(N)$ (skewed tree).
80	a	Best case: $O(1)$ (first node / root position).
81	b	Average case: $O(\lg N)$.
82	b	5 is inserted first, so it becomes the root.
83	a	$7 > 5 \rightarrow$ right to 9; $7 < 9 \rightarrow$ left child of 9.
84	a	$20 > 5 \rightarrow$ 9; $20 > 9 \rightarrow$ 12; $20 > 12 \rightarrow$ right child of 12.
85	b	The slide labels the chain General case 2, General case 1, Base case.
86	c	Sorted input degenerates into a linear chain (input (c) A B C D E F G).
87	b	Input (a) D B F A C E G gives the short balanced tree.
88	b	The same items give very different heights depending on insertion order.
89	b	"Calculate the number of nodes in the tree recursively".
90	b	If tree is empty (base case) length = 0.
91	b	Nodes in left subtree + nodes in right subtree + 1.
92	a	<code>return CountNodes(tree->left) + CountNodes(tree->right) + 1;</code>
93	a	<code>int TreeType::LengthIs() { return CountNodes(root); }</code>
94	c	$O(N)$.
95	b	Counting requires visiting every node, so no branch can be skipped.
96	b	$0 + 1 + 1 = 2$ — the slide shows 2 next to node 3.
97	b	The final answer at the root is 9.
98	b	"Find the target node recursively".
99	b	Base case 1: tree is empty $\Rightarrow$ item is not found.
100	b	Base case 2: item == current_node_info $\Rightarrow$ item is found.
101	b	General case 1: search the left subtree.
102	a	<code>item = tree->info; found = true;</code>
103	a	<code>void Retrieve(TreeNode* tree, ItemType& item, bool& found)</code>
104	b	Retrieve only reads the tree, so a copy of the pointer is enough.
105	a	Worst $O(N)$, best $O(1)$, average $O(\log N)$.
106	b	One comparison at the root is enough.
107	a	Slide labels: General case 2, General case 1, General case 2, Base case 2.
108	b	10 is absent; the walk falls off the tree and hits base case 1.
109	b	"Print the items in the tree (in sorted order) recursively".
110	b	Left subtree, current node, right subtree.
111	b	Nothing to print.
112	b	<code>if (tree != NULL)</code>
113	b	Left-node-right is exactly inorder.
114	b	3 4 5 6 7 8 9 12 20.
115	c	$O(N)$ — every node is printed.
116	c	Four: leaf, left == NULL, right == NULL, and the two-children else branch.
117	b	<code>tree = NULL; delete tempPtr;</code>
118	a	<code>tree = tree->right; delete tempPtr;</code>
119	b	<code>tree = tree->left; delete tempPtr;</code>
120	b	"Replace it with its predecessor (rightmost node in the left subtree)".
121	b	One step left, then right as far as possible.
122	b	<code>while (tree->right != NULL) tree = tree->right; data = tree->info;</code>

123	b	Delete(tree->left, data); removes the duplicated predecessor.
124	b	The rightmost node of 9's left subtree (3 → 4) is 4.
125	b	The rightmost node of 16's left subtree (10 → 14) is 14.
126	a	A one-child node is bypassed: the parent link is redirected to the single child.
127	b	tempPtr keeps the address so the node can be freed after the pointer is re-linked.
128	c	The else branch (key matches) calls DeleteNode(tree);
129	a	void TreeType::DeleteItem(ItemType item) { Delete(root, item); }
130	b	Left subtree, then the node, then the right subtree.
131	a	Left subtree, right subtree, then the node.
132	b	The node, then left subtree, then right subtree.
133	b	3 4 9 10 12 13 14 16 18 20.
134	a	4 3 12 13 14 10 20 18 16 9.
135	b	9 3 4 16 10 14 13 12 18 20.
136	b	Inorder on a BST yields ascending keys.
137	b	QueType preQue; QueType inQue; QueType postQue;
138	b	InOrder(left); Enqueue(info); InOrder(right);
139	a	Enqueue first, then PreOrder(left), PreOrder(right).
140	c	PostOrder(left); PostOrder(right); Enqueue(info);
141	b	switch (order) with PRE_ORDER / IN_ORDER / POST_ORDER cases.
142	b	finished = false; is the first statement.
143	b	e.g. inQue.Dequeue(item); if (inQue.IsEmpty()) finished = true;
144	b	"Not necessarily maintains the First In First Out order".
145	b	"Items can be added in any order".
146	b	"Always, the item with the highest priority is deleted".
147	b	"The priority queue is arranged to support access to the highest-priority item".
148	b	throw FullPQ();
149	b	throw EmptyPQ();
150	b	Highest priority element removed; item is a copy of the removed element.
151	b	A heap is a complete binary tree obeying an order property.
152	b	Min-heap: value ≤ value of each child.
153	b	Max-heap: value ≥ value of each child.
154	b	The slide labels "A complete binary tree" as the Shape property.
155	b	The slide labels the parent-child comparison as the Order property.
156	b	"The shape of all heaps with a given number of elements is the same".
157	b	"The root node always contains the largest value in the heap".
158	b	"in addition, the subtrees are heaps as well".
159	b	A heap orders parent vs child only; siblings are unordered, so it is not a BST.
160	c	Only parent-child order is guaranteed, so inorder output is not sorted.
161	b	"the order property is violated by the root node only".
162	a	"move the element down from the root position until ... the order property is satisfied".
163	b	"the order property is violated by the last leaf node only".
164	b	"move the element up in the tree until it ends up in its correct position".
165	b	6 swaps with the larger child 11.
166	b	Then 6 swaps with 9, and finally with 8.
167	b	maxChild is chosen, i.e. the larger child in a max-heap.

168	a	11 swaps with 6, then 8, then 9, ending at the root.
169	b	$O(\log N)$.
170	b	$O(\log N)$.
171	b	Each step moves one level, and a complete tree has $\sim \log_2 N$ levels.
172	a	Swap is three assignments $\Rightarrow O(1)$.
173	a	<code>temp = one; one = two; two = temp;</code>
174	b	<code>if (bottom > root)</code>
175	b	<code>parent = (bottom-1) / 2;</code>
176	a	<code>if (elements[parent] < elements[bottom])</code>
177	b	<code>ReheapUp(root, parent);</code>
178	a	<code>int maxChild, rightChild = root*2+2, leftChild = root*2+1;</code>
179	b	<code>leftChild ≤ bottom</code> means at least one child exists.
180	b	"it is the only child".
181	a	If <code>left ≤ right</code> then <code>maxChild = rightChild</code> else <code>maxChild = leftChild</code> .
182	a	<code>if (elements[root] < elements[maxChild])</code>
183	b	<code>ReheapDown(maxChild, bottom);</code>
184	b	"since the tree is complete, there are not any holes in the tree".
185	b	"Root at index 0".
186	b	Left child at index $2i+1$.
187	c	Right child at index $2i+2$.
188	b	Parent at index $(i-1)/2$.
189	b	Children of index 1 are indices 3 and 4 $\Rightarrow$ 7 and 8.
190	a	$(6-1)/2 = 2$ with integer division.
191	a	<code>ItemType* elements; int numElements;</code>
192	b	<code>void ReheapDown(int root, int bottom); void ReheapUp(int root, int bottom);</code>
193	b	<code>template<class ItemType> class HeapType</code>
194	a	<code>class FullPQ{}; class EmptyPQ{};</code>
195	a	<code>int length; HeapType<ItemType> items; int maxItems;</code>
196	b	<code>PQType(int);</code>
197	a	<code>maxItems = max; items.elements = new ItemType[max]; length = 0;</code>
198	b	<code>delete [] items.elements;</code> — array form.
199	b	<code>length = 0;</code>
200	b	<code>return length == maxItems;</code>
201	b	<code>return length == 0;</code>
202	a	All three are $O(1)$.
203	b	Increment length, store at length-1, then <code>ReheapUp</code> .
204	a	<code>items.ReheapUp(0, length-1);</code>
205	b	<code>if (length == maxItems) throw FullPQ();</code>
206	b	$O(\log N)$, dominated by <code>ReheapUp</code> .
207	c	length becomes 11, so the item goes to index <code>length-1 = 10</code> .
208	b	15 swaps with 8, then 9, then 11.
209	b	15 becomes the new root.
210	a	Replace root with last leaf, decrement length, <code>ReheapDown</code> .
211	b	<code>item = items.elements[0];</code> — the root holds the highest priority.
212	a	<code>items.ReheapDown(0, length-1);</code>

213	a	Save root, copy last leaf to root, decrement length, ReheapDown(0, length-1).
214	b	$O(\log N)$, dominated by ReheapDown.
215	b	The root 11 is the highest priority item.
216	b	The last leaf 6 is moved to the root.
217	b	$10 - 1 = 9$.
218	a	9 8 4 7 6 3 1 2 5 after 6 sinks to index 4.
219	b	After the decrement the old last slot is outside the heap and must not be touched.
220	b	<code>#include "heap.h"</code>
221	a	Both InsertItem and RetrieveItem are $O(\log N)$ on average.
222	b	LengthIs and Print must visit every node.
223	b	IsEmpty is a single pointer comparison.
224	b	A skewed tree behaves like a linked list.
225	b	A heap is always complete, so its height is always $\sim \log_2 N$.